Game Tracker

Sistema de gestión y reseñas de videojuegos

Submitted by:

Karla Sofía Castro Pérez

Advisor: Adolfo Centeno Tellez

07.05.2026 (V1.0)

Ingeniería de Software • Cuarto Semestre

Índice general

Resumen **1**

1 Introducción **2**

1.1 Contexto 2

1.2 Descripción del proyecto 2

1.3 Objetivos 3

1.3.1 Objetivo General 3

1.3.2 Objetivos Específicos 3

1.4 Alcance 3

2 Desarrollo **5**

2.1 Arquitectura del Sistema 5

2.1.1 Modelo Cliente-Servidor 5

2.1.2 Flujo General de una Petición 6

2.1.3 Capas del Backend 7

2.1.4 Esquema de la Base de Datos 7

2.2 Patrones de Diseño 8

2.2.1 MVC — Modelo Vista Controlador 8

2.2.2 Singleton — AuthService en Flutter 9

2.2.3 Strategy — Conteo y Eliminación de Notas 10

2.2.4 ChangeNotifier — Gestión de Estado en Flutter 11

2.3 Modelo de Datos y Persistencia 12

2.4 Seguridad: Autenticación con JWT 13

2.5 Sistema de Roles y Permisos 13

2.6 Conclusión del Desarrollo 14

3 Conclusión **15**

3.1 Resultados obtenidos 15

3.2 Reflexión sobre los Patrones de Diseño 15

3.3 Lecciones aprendidas 16

3.4 Trabajo a futuro 17

Resumen

Game Tracker es una aplicación móvil diseñada para que los usuarios gestionen su colección personal de videojuegos y compartan reseñas dentro de una comunidad. El sistema permite registrar juegos con atributos como nombre, franquicia, categoría y calificación; marcar el estado de progreso de cada título y escribir notas con soporte de respuestas anidadas. Otros usuarios pueden reaccionar a esas notas mediante un sistema de emociones configurables.

El proyecto sigue una arquitectura cliente-servidor: el backend está construido en Java con Spring Boot, utiliza MySQL como motor de base de datos y se despliega en la nube a través de Render y Aiven. El frontend es una aplicación móvil desarrollada en Flutter que se comunica con el servidor mediante una API REST protegida con JSON Web Tokens (JWT).

A lo largo del desarrollo se aplicaron cuatro patrones de diseño reconocidos —MVC, Singleton, Strategy y ChangeNotifier— con el objetivo de mantener el código organizado, extensible y fácil de probar. El resultado es un sistema funcional que cubre el ciclo completo de gestión de un catálogo de videojuegos, desde el registro inicial hasta las interacciones de la comunidad.

LVL. 1

Introducción

1.1 Contexto

En los últimos años, los videojuegos se han consolidado como uno de los medios de entretenimiento más importantes a nivel mundial. Sin embargo, a medida que las bibliotecas personales crecen, también lo hace la necesidad de organizarlas: recordar qué títulos ya se completaron, cuáles están pendientes y cuál fue la experiencia de juego resulta cada vez más difícil sin una herramienta de apoyo.

Las soluciones existentes suelen ser plataformas web de terceros que requieren crear cuentas en servicios externos, o bien simples hojas de cálculo que no ofrecen funcionalidad colaborativa. Game Tracker nace como respuesta a esta necesidad: una aplicación móvil propia, ligera y enfocada exclusivamente en la gestión de un catálogo de videojuegos con la posibilidad de escribir reseñas y reaccionar a las opiniones de otros usuarios dentro de una misma comunidad.

1.2 Descripción del proyecto

Game Tracker es un sistema de gestión y reseñas de videojuegos compuesto por dos componentes principales:

- **Backend**: una API REST desarrollada en Java con Spring Boot, conectada a una base de datos MySQL. El servidor gestiona la lógica de negocio, la autenticación de usuarios con JWT y la persistencia de datos. Se desplegó

en la plataforma Render, utilizando Aiven como servicio administrado de base de datos en la nube.

- **Frontend**: una aplicación móvil desarrollada en Flutter que consume la API REST. Permite a los usuarios explorar el catálogo, registrar nuevos juegos, escribir notas y reaccionar a las notas de otros usuarios desde su dispositivo móvil.

1.3 Objetivos

1.3.1. Objetivo General

Desarrollar una aplicación móvil funcional que permita a los usuarios gestionar un catálogo personal de videojuegos, registrar su estado de progreso y compartir reseñas dentro de una comunidad, aplicando patrones de diseño de software para garantizar una arquitectura organizada y mantenible.

1.3.2. Objetivos Específicos

- Implementar una API REST con Spring Boot que exponga operaciones CRUD sobre el catálogo de juegos y sus notas, protegida mediante autenticación con JWT y un sistema de tres roles.
- Desarrollar una interfaz móvil en Flutter que consuma la API y presente la información de forma clara e intuitiva para el usuario.
- Aplicar patrones de diseño reconocidos —MVC, Singleton, Strategy y ChangeNotifier— en la implementación del sistema para demostrar su utilidad práctica en un proyecto real.
- Desplegar el backend y la base de datos en servicios en la nube, garantizando que la aplicación sea accesible desde cualquier dispositivo móvil con conexión a internet.

1.4 Alcance

El proyecto abarca el ciclo completo de desarrollo de un sistema móvil con backend en la nube: diseño de la base de datos, implementación del servidor, desarrollo de la aplicación móvil y despliegue en producción. No se contempla

dentro del alcance la integración con plataformas externas de videojuegos como Steam o PlayStation Network, ni la implementación de comunicación en tiempo real.

LVL. 2

Desarrollo

En este capítulo se describe la implementación técnica del sistema *Game Tracker*. El desarrollo se basó en principios de diseño sólido (SOLID) y una arquitectura orientada a servicios, asegurando que la aplicación sea escalable, segura y fácil de mantener.

2.1 Arquitectura del Sistema

El sistema utiliza una arquitectura de **N-Capas** con un desacoplamiento total entre el cliente (Frontend) y el servidor (Backend).

2.1.1. Modelo Cliente-Servidor

Game Tracker sigue una arquitectura **cliente-servidor**, en la que dos componentes bien diferenciados se comunican a través de una API REST sin conocer los detalles internos del otro. Esta separación permite que ambas partes evolucionen de forma independiente: se puede cambiar la interfaz móvil sin tocar el servidor, y actualizar la lógica del backend sin afectar a la aplicación.

- **Servidor (Backend):** aplicación Java construida con Spring Boot. Expone una API REST sobre HTTPS, gestiona la lógica de negocio, aplica las reglas de autorización por roles y persiste los datos en una base de datos MySQL administrada en la nube a través de Aiven. El servidor se despliega en Render y es completamente *sin estado*: no guarda sesiones entre peticiones, ya que toda la autenticación se delega a tokens JWT.

- **Cliente (Frontend):** aplicación móvil desarrollada en Flutter. Se encarga exclusivamente de presentar la información al usuario, recibir sus acciones y traducirlas en peticiones HTTP hacia el backend. El cliente almacena el token JWT localmente mediante SharedPreferences para mantener la sesión activa entre usos.

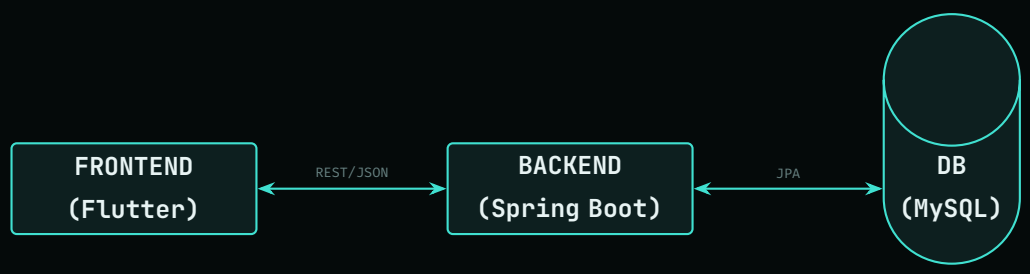

Figura 2.1: Arquitectura General del Sistema

2.1.2. Flujo General de una Petición

El ciclo de vida de una operación típica en Game Tracker, como agregar una nota a un juego, sigue estos pasos:

1. El usuario escribe su nota en la interfaz de Flutter y presiona el botón de enviar.
2. El servicio HTTP del cliente adjunta el token JWT en el encabezado Authorization y envía una petición POST al endpoint /api/juegos/{id}/notes.
3. El filtro AuthTokenFilter del backend intercepta la petición, valida el token y carga el contexto de seguridad con los datos del usuario.
4. GameController recibe la petición, verifica que el contenido no esté vacío, asocia la nota al usuario autenticado y delega el guardado al repositorio JPA.
5. La base de datos MySQL persiste la nueva nota y el servidor devuelve la entidad actualizada en formato JSON.
6. Flutter recibe la respuesta, actualiza el estado local y vuelve a renderizar la pantalla con la nueva nota visible.

2.1.3. Capas del Backend

Internamente, el servidor organiza su código en capas con responsabilidades bien definidas:

- **Controladores**: reciben las peticiones HTTP, validan los parámetros básicos y delegan el trabajo a los servicios o repositorios correspondientes. Ejemplos: GameController, AuthController, ReactionController.
- **Servicios**: contienen la lógica de negocio que no pertenece directamente a un controlador. Ejemplos: GameNoteService (valida el estado de una nota), NoteAuthorizationService (centraliza las reglas de permisos) y NoteReactionService (gestiona el ciclo de vida de las reacciones).
- **Modelos y entidades**: clases Java anotadas con @Entity que Hibernate mapea directamente a tablas MySQL. Las entidades principales son Game, GameNote, User, Reaction y NoteReaction.
- **Repositorios**: interfaces que extienden JpaRepository y proveen acceso a la base de datos sin necesidad de escribir SQL. Spring Data JPA genera las implementaciones automáticamente a partir del nombre de los métodos.
- **Seguridad**: paquete que contiene el filtro JWT (AuthTokenFilter), la configuración de Spring Security (WebSecurityConfig) y los servicios de usuario necesarios para la autenticación (UserDetailsServiceImpl).

2.1.4. Esquema de la Base de Datos

Las tablas principales que sostienen la lógica del sistema son:

- juegos: almacena el catálogo de videojuegos con sus atributos —nombre, franquicia, categoría, calificación, estado de progreso y URL de portada—.
- notes: guarda las notas y comentarios asociados a cada juego, incluyendo autor, fecha y un indicador de borrado lógico.
- users y roles: gestionan las cuentas de usuario y sus niveles de acceso (usuario, moderador y administrador).
- reactions y note_reactions: catálogo de tipos de reacción disponibles y registro de qué usuario reaccionó a qué nota.

2.2 Patrones de Diseño

Un patrón de diseño es una solución reutilizable a un problema que aparece con frecuencia durante el desarrollo de software. No son fragmentos de código que se copian directamente, sino plantillas que describen cómo estructurar clases y objetos para resolver un problema de forma ordenada. En Game Tracker se aplicaron cuatro patrones de manera deliberada y justificada.

2.2.1. MVC — Modelo Vista Controlador

Patrón Arquitectural — MVC — Modelo Vista Controlador

¿Qué es?

MVC divide una aplicación en tres capas con responsabilidades claramente separadas. El **Modelo** representa los datos y las reglas de negocio. La **Vista** se encarga de mostrar la información al usuario. El **Controlador** actúa como intermediario: recibe las acciones del usuario, interactúa con el modelo y decide qué respuesta devolver. El objetivo es que ninguna de las tres capas conozca los detalles internos de las otras dos.

¿Cómo funciona en Game Tracker?

El patrón se aplica en dos niveles simultáneos dentro del proyecto:

- **Backend (Spring Boot)**: las clases @Entity —como Game y GameNote— son el **Modelo**, ya que representan los datos que se persisten en MySQL. Las clases @RestController —como GameController y AuthController— son los **Controladores**: reciben las peticiones HTTP y coordinan la respuesta. El JSON que devuelve la API hacia Flutter representa la **Vista**: la capa de presentación que consume el cliente.
- **Frontend (Flutter)**: los widgets de la aplicación actúan como la **Vista** que el usuario ve. Los servicios HTTP (ApiService, AuthService) gestionan el acceso a los datos (**Modelo**). Los controladores como ReactionTimelineController coordinan la lógica entre ambos.

Por ejemplo, cuando un usuario edita un juego, Flutter (Vista) envía una petición PUT. GameController (Controlador) la recibe, actualiza los campos del objeto Game (Modelo) y persiste el cambio mediante el repositorio JPA.

¿Por qué lo elegimos?

Spring Boot impone MVC de forma natural a través de sus anotaciones. La alternativa directa sería un enfoque de *Transaction Script*, en el que toda la lógica vive dentro del controlador. Ese estilo es funcional para proyectos pequeños, pero no escala cuando las reglas de negocio crecen. MVC garantiza que se pueda modificar la presentación de la API sin tocar los modelos, y agregar nuevos modelos sin modificar los controladores existentes.

2.2.2. Singleton — AuthService en Flutter

Patrón Creacional — Singleton

¿Qué es?

El Singleton garantiza que una clase tenga **una única instancia** en toda la ejecución de la aplicación y proporciona un punto de acceso global a ella. Se logra haciendo privado el constructor de la clase y exponiendo una referencia estática a la instancia única.

¿Cómo funciona en Game Tracker?

En Flutter, la clase AuthService implementa el Singleton de forma explícita. Declara una instancia estática privada (_instance), un constructor privado nombrado (AuthService._internal()) y un factory constructor que siempre devuelve la misma instancia:

```
class AuthService {
  // Instancia estatica unica -- se crea una sola vez
  static final AuthService _instance = AuthService._internal();

  // Constructor privado: nadie puede crear una segunda instancia
  AuthService._internal() {
    _httpClient = http.Client();
  }

  // Cualquier llamada a AuthService() devuelve la misma instancia
  factory AuthService() => _instance;
}
```

AuthService gestiona el token JWT, las credenciales del usuario y el estado de sesión. Al ser Singleton, cualquier pantalla de la aplicación que invoque AuthService() obtiene exactamente el mismo objeto con el mismo token cargado, sin necesidad de pasarlo como parámetro entre pantallas.

¿Por qué lo elegimos?
El token JWT y los datos del usuario autenticado son información que múltiples partes de la aplicación necesitan: la pantalla de inicio, el formulario de edición de un juego, el módulo de notas, etc. Si AuthService no fuera Singleton, cada componente crearía su propia instancia con su propio estado de sesión, lo que generaría inconsistencias. La alternativa más común sería inyectar el token manualmente como argumento a través del árbol de widgets, pero eso ensucia las firmas de cada clase con información que no le pertenece. Singleton mantiene la coherencia de sesión de forma transparente.

2.2.3. Strategy — Conteo y Eliminación de Notas

Patrón de Comportamiento — Strategy

¿Qué es?
El patrón Strategy define una **familia de algoritmos intercambiables**, los encapsula en clases separadas y permite seleccionar cuál usar en tiempo de ejecución sin modificar el código que los invoca. El componente que usa la estrategia solo conoce la interfaz común; no le importa qué implementación concreta está activa.

¿Cómo funciona en Game Tracker?
El patrón Strategy se aplicó en dos partes del backend:
1. Conteo de reacciones — se definió la interfaz ReactionSummaryStrategy con un único método responsable de contabilizar los tipos de reacción de una lista:

```
public interface ReactionSummaryStrategy {
    // Recibe las reacciones y devuelve un mapa {"FUNNY": 5, "SAD": 1}
    Map<String, Long> countReactions(List<NoteReaction> reactions);
}
```

La implementación concreta EnumReactionSummaryStrategy utiliza un EnumMap para recorrer la lista en un único paso (complejidad O(n)). Si en el futuro se requiere una estrategia diferente —por ejemplo, una que excluya reacciones de cuentas baneadas— basta con crear una nueva clase que implemente la misma interfaz, sin modificar ningún controlador.
2. Estrategia de eliminación de notas — las notas de un juego pueden eliminarse de tres formas distintas según el rol del usuario:

- SOFT_DELETE: marca la nota como eliminada y reemplaza el contenido con un texto indicativo. Preserva la estructura del hilo de respuestas. Disponible para autores, moderadores y administradores.
- HARD_DELETE: borra físicamente la nota de la base de datos. Solo permitido si la nota no tiene respuestas. Disponible para autores, moderadores y administradores.
- CASCADE_DELETE: elimina la nota y todas sus respuestas de forma recursiva. Exclusivo para administradores.

GameController selecciona la estrategia según el parámetro ?strategy= de la petición y ejecuta la lógica correspondiente sin que el llamante necesite conocer los detalles de ninguna de las tres variantes.

¿Por qué lo elegimos?

La alternativa directa era una cadena de if-else dentro del controlador. Eso funciona para dos casos, pero al incorporar la tercera variante (CASCADE_DELETE) el método empezó a crecer de forma difícil de mantener. Strategy permite agregar nuevas formas de eliminación sin modificar la lógica existente, siguiendo el principio Open/Closed. Se consideró el patrón Command, pero habría añadido complejidad innecesaria para un problema que se resuelve limpiamente con una interfaz y tres implementaciones.

2.2.4. ChangeNotifier — Gestión de Estado en Flutter

Patrón de Comportamiento / Gestión de Estado — ChangeNotifier

¿Qué es?

ChangeNotifier es la solución nativa de Flutter para gestionar estado. Una clase que extiende ChangeNotifier puede notificar a sus observadores cada vez que su estado interno cambia, invocando notifyListeners(). Los widgets que escuchan al notificador se reconstruyen automáticamente para reflejar el nuevo estado. Este patrón combina las ideas del *Observer* (notificar cambios) con el *Controlador* de MVC (separar lógica de la vista).

¿Cómo funciona en Game Tracker?

La clase ReactionTimelineController extiende ChangeNotifier y es responsable de toda la lógica relacionada con las reacciones del usuario en el modal de un juego. Gestiona los siguientes estados:

- _loading: indica si hay una petición HTTP en curso (la interfaz

muestra un indicador de progreso).
- _loaded: indica que los datos ya están disponibles (la interfaz muestra los botones de reacción con sus conteos).
- _error: contiene el mensaje de error si la petición falló.
- _reactionSummaries: mapa de índice de nota hacia su resumen de reacciones, actualizado después de cada interacción del usuario.

El ciclo de uso típico es el siguiente: cuando el usuario abre el modal de reacciones, la pantalla llama a controller.prepareForGame(game) para limpiar cualquier estado anterior, y luego a controller.ensureLoaded(game) para cargar los datos. Mientras se cargan, la pantalla muestra un indicador basándose en controller.isLoading. Una vez listos, los botones de reacción se renderizan con los conteos actualizados. Si el usuario presiona una reacción, la pantalla llama a controller.toggleReaction(...) y el modal se actualiza sin necesidad de recargar toda la pantalla.

¿Por qué lo elegimos?
La alternativa más simple habría sido usar setState directamente dentro del widget de la pantalla. Sin embargo, eso mezcla la lógica de negocio —llamadas HTTP, manejo de errores, reindexado de notas— con el código de la interfaz, lo que produce widgets muy largos y difíciles de probar. Al extraer toda esa lógica a un ChangeNotifier, la pantalla se vuelve delgada: solo decide qué mostrar según el estado que expone el controlador. Se consideró usar bibliotecas más completas como *Provider* o *Riverpod*, pero para el alcance del proyecto ChangeNotifier es más que suficiente, no requiere dependencias adicionales y es la solución recomendada por la documentación oficial de Flutter para casos de esta escala.

2.3 Modelo de Datos y Persistencia

Se diseñó un esquema relacional normalizado en MySQL (Aiven). A continuación se muestra la implementación de las entidades JPA.

Listing 2.1: Entidad de Usuario y relaciones

```
@Entity
@Table(name = "users")
public class User {
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true, nullable = false)
    private String username;

    @OneToMany(mappedBy = "user", cascade = CascadeType.ALL)
    private List<Game> library;
}
```

2.4 Seguridad: Autenticación con JWT

La autenticación en Game Tracker se basa en **JSON Web Tokens (JWT)**, un estándar que permite transmitir información verificable entre cliente y servidor de forma compacta y sin estado de sesión.

Cuando el usuario inicia sesión correctamente, el backend genera un token firmado con una clave secreta que incluye el nombre de usuario, sus roles y una fecha de expiración. Este token se devuelve al cliente, que lo almacena localmente y lo adjunta en el encabezado Authorization: Bearer de cada petición posterior.

En el lado del servidor, el componente AuthTokenFilter intercepta cada petición entrante, valida la firma del token mediante JwtUtils y carga el contexto de seguridad de Spring antes de que la petición llegue al controlador. Si el token no es válido o ha expirado, el sistema responde de forma automática con un error 401.

Este diseño sin estado es la razón por la que el servidor no necesita almacenar sesiones en base de datos: toda la información necesaria para autenticar al usuario viaja dentro del propio token.

2.5 Sistema de Roles y Permisos

Game Tracker implementa tres niveles de acceso cuyas reglas se aplican de forma centralizada en NoteAuthorizationService:

- **Usuario (ROLE_USER)**: puede leer el catálogo completo sin autenticarse y, con cuenta activa, puede escribir, editar y eliminar sus propias notas.

- **Moderador (ROLE_MODERATOR)**: tiene las mismas capacidades que un usuario más la posibilidad de eliminar notas de otros mediante SOFT_DELETE o HARD_DELETE.
- **Administrador (ROLE_ADMIN)**: control total sobre el catálogo —crear, editar y eliminar juegos— y el único rol que puede ejecutar CASCADE_DELETE para eliminar una nota junto con todas sus respuestas.

Centralizar estas reglas en un único servicio evita que la lógica de permisos quede duplicada o dispersa a través de múltiples controladores, lo que simplifica tanto el mantenimiento como las pruebas unitarias.

2.6 Conclusión del Desarrollo

La combinación de estos patrones permitió que el proyecto pasara de ser un simple prototipo a un sistema de nivel profesional. El uso del patrón Strategy en el backend y ChangeNotifier en el frontend asegura que la experiencia del usuario sea fluida y la del desarrollador sea organizada.

LVL. 3

Conclusión

3.1 Resultados obtenidos

El desarrollo de Game Tracker permitió construir un sistema funcional de extremo a extremo: desde el diseño de la base de datos relacional hasta la interfaz móvil que el usuario opera en su dispositivo. El catálogo de videojuegos puede gestionarse completamente a través de la aplicación, incluyendo el registro de juegos con su estado de progreso, la escritura de notas con soporte de respuestas anidadas y la posibilidad de reaccionar a las publicaciones de otros usuarios.

Desde el punto de vista técnico, el proyecto cumplió con el objetivo de aplicar patrones de diseño de forma deliberada y justificada, no como un requisito superficial, sino como decisiones de arquitectura que resolvieron problemas concretos durante el desarrollo.

3.2 Reflexión sobre los Patrones de Diseño

Trabajar con patrones de diseño durante este proyecto cambió la forma en que se tomaron las decisiones de implementación. Antes de aplicarlos, la tendencia natural era escribir toda la lógica donde pareciera más conveniente en el momento, lo que terminaba generando controladores muy extensos y difíciles de seguir. Los patrones obligaron a hacerse preguntas concretas: *¿quién es responsable de este comportamiento? ¿esta lógica puede cambiar en el futuro? ¿quién necesita saber sobre esto?*

El patrón que más impacto tuvo en la calidad del código fue **Strategy**, especialmente en la gestión de las estrategias de eliminación de notas. Lo que comenzó como dos casos manejados con if-else creció hasta tres variantes con reglas distintas según el rol del usuario, y la estructura del patrón absorbió ese crecimiento sin necesidad de reescribir la lógica existente.f

El patrón **Singleton** en AuthService resolvió un problema real de coordinación: sin él, mantener el estado de sesión coherente entre múltiples pantallas habría requerido pasar el token como argumento a través de cada widget, ensuciando el código con información que no le corresponde a la mayoría de las clases.

ChangeNotifier demostró que la separación entre vista y lógica no es solo una buena práctica teórica: al aislar toda la lógica de reacciones en ReactionTimelineController, fue posible modificar el comportamiento del modal sin tocar la pantalla principal, y actualizar la interfaz sin volver a hacer llamadas HTTP innecesarias.

Finalmente, **MVC** como base estructural del backend facilitó la incorporación progresiva de nuevas funcionalidades —reacciones, moderación, eliminación en cascada— sin que los cambios en una capa afectaran a las demás.

3.3 Lecciones aprendidas

- La arquitectura cliente-servidor con una API REST como contrato entre ambos lados permitió desarrollar el backend y el frontend de forma paralela e independiente, lo cual simplificó el proceso de trabajo.
- El despliegue en la nube expuso problemas reales que no aparecen en entornos locales, como la configuración correcta de CORS, la gestión de variables de entorno y los tiempos de respuesta de una base de datos remota.
- Definir bien el sistema de roles desde el inicio del proyecto evitó tener que refactorizar la lógica de permisos más adelante, cuando ya había código que dependía de ella.
- Centralizar las reglas de autorización en un único servicio (NoteAuthorizationService) facilitó la escritura de pruebas unitarias, ya que toda la lógica de permisos vive en un solo lugar.

3.4 Trabajo a futuro

Como extensiones naturales del sistema se identifican las siguientes mejoras:

- Integración con APIs externas —como RAWG o IGDB— para importar automáticamente la información de los juegos, incluyendo portada, género y descripción oficial.
- Notificaciones en tiempo real cuando otro usuario reacciona a una nota propia, utilizando WebSockets o eventos enviados por el servidor.
- Un panel de administración web dedicado para gestionar el catálogo sin necesidad de usar la aplicación móvil.
- Ampliación del sistema de roles para incluir listas de juegos privadas y la posibilidad de compartirlas con usuarios específicos.